

**МИНИСТЕРСТВО НАУКИ И ОБРАЗОВАНИЯ АЗЕРБАЙДЖАНСКОЙ
РЕСПУБЛИКИ**

**АЗЕРБАЙДЖАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
НЕФТИ И ПРОМЫШЛЕННОСТИ**

КАФЕДРА: «КОМПЬЮТЕРНАЯ ИНЖЕНЕРИЯ»

**ПРЕДМЕТ:
«ВЕБ-СИСТЕМЫ И ТЕХНОЛОГИИ»**

КУРСОВАЯ РАБОТА

Тема: «Разработка системы онлайн-бронирования отелей»

Группа: 680.23

Студент: Мухтиев Саид

Преподаватели: Халилов Матлаб, Алиев Джамиль

БАКУ 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
1.1. Что представляет собой данная система.....	4
1.2. HTML — структура страниц.....	5
1.3. CSS — оформление и внешний вид	6
1.4. JavaScript — логика и интерактивность.....	8
1.5. Как всё это устроено вместе	9
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	10
2.1. HTML-разметка страниц	10
2.2. CSS-стили.....	13
2.3. JavaScript-логика	16
ГЛАВА 3. РЕЗУЛЬТАТЫ РАБОТЫ	21
3.1. Главная страница.....	21
3.2. Каталог отелей.....	22
3.3. Страница бронирования	24
3.4. Страница контактов	26
3.5. Итоговая оценка результатов.....	27
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	29

ВВЕДЕНИЕ

Онлайн-бронирование отелей давно стало частью нашей повседневной жизни. Сегодня большинство гостей предпочитают не звонить на ресепшн, а выбирать и бронировать номера прямо через сайт. При этом качество такого сайта напрямую влияет на то, закажет ли пользователь бронь или просто покинет сайт. Вот почему разработка грамотного интерфейса системы бронирования является актуальной и практически значимой задачей.

Целью курсовой работы является создание сайта системы онлайн-бронирования отелей с использованием HTML, CSS и JavaScript. Если не знать принципы этих 3 технологий, то дальнейшее изучение серверной разработки станет крайне тяжёлой задачей. Серверные технологии и базы данных в проекте мы не применяем, так как вся логика работает прямо в браузере: сайт позволяет просматривать каталог отелей, фильтровать их по разным параметрам, заполнять форму бронирования и получать подтверждение.

Задачи работы:

- Изучить применение HTML, CSS и JavaScript при создании сайта.
- Спроектировать структуру сайта из нескольких страниц.
- Написать разметку, стили и логику для каждой страницы.
- Реализовать фильтрацию отелей и валидацию форм на языке JavaScript.
- Проверить правильность работы сайта, а также зафиксировать результаты.

Сайт будет состоять из 4 страниц: «Главная», «Каталог отелей», «Страница бронирования» и «Контакты». На каждой из них реализована своя логика, а стиль выглядит одинаково единым для всего сайта.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Что представляет собой данная система

Система бронирования отелей представляет собой веб-приложение, в котором пользователь может найти подходящий отель, ознакомиться с информацией об этом самом отеле, а также оформить заявку на проживание. Обычно подобные сервисы работают с сервером и базой данных, но в данной работе всё реализовано только на стороне клиента: данные об отелях хранятся в JavaScript-массиве, а подтверждение бронирования имитируется через модальное окно.

Здесь пользователь проходит следующий путь: открывает главную страницу, нажимает кнопку «Найти отель», далее переходит в каталог, выбирает подходящий вариант, заполняет форму бронирования и получает подтверждение. Ещё пользователю доступна страница с контактами и формой обратной связи. Тем самым, мы наглядно видим, что каждая отдельная страница отвечает за конкретный шаг пользователя и не перегружает остальные части сайта лишним содержанием.

Как отмечает Спасов Д.Ю. в своей научной работе по разработке веб-приложений для гостиничного бизнеса, современная система бронирования является важным инструментом в гостиничной индустрии. Она упрощает процесс управления гостиничными ресурсами и обеспечивает удобство для гостей, а также способствует увеличению доходов и улучшению качества обслуживания. С распространением интернет-технологий компании гостиничного сектора всё активнее переходят на онлайн-продажи, поскольку отсутствие IT-решения становится одним из главных препятствий для роста продаж.

1.2. HTML — структура страниц

HTML (HyperText Markup Language) — это язык разметки, с помощью которого браузер описывает содержимое страницы. Стоит отметить, что HTML конкретно языком программирования НЕ является, это именно разметка: это где находятся заголовок, абзац и форма с полями. Текущий стандарт HTML5 был принят в 2014-ом году и добавил много удобных семантических тегов.

В работе HTML задаёт структуру каждой из 4 страниц. Страницы связаны между собой навигационным меню, которое выглядит одинаково на всех из них. Достигается это просто: один и тот же блок навигации присутствует в каждом файле.

Семантические теги

В HTML5 есть теги, которые не просто оборачивают контент, но и указывают, что именно внутри. Тег `<header>` используется для шапки с навигацией, `<nav>` — для меню ссылок, `<section>` — для смысловых блоков на странице, `<footer>` — для подвала (то есть для нижней части сайта). Именно эти теги и помогают браузерам и поисковым системам понимать структуру документа.

Формы

Основная интерактивная часть сайта — это формы. На странице бронирования есть поля для имени, фамилии, email, телефона, выбора отеля и дат. Для дат используется `<input type="date">`, и браузер автоматически отображает виджет выбора даты. Поле email имеет `type="email"`, что обеспечивает базовую проверку формата ещё до JavaScript. Атрибут `placeholder` подсказывает пользователю, что нужно ввести.

Тег	Назначение	Где используется в проекте
<header>	Шапка страницы	Логотип и навигация
<nav>	Блок навигации	Меню ссылок между страницами
<section>	Тематический раздел	Неро, преимущества, каталог
<input>	Поле ввода	Формы поиска и бронирования
<footer>	Подвал страницы	Копирайт и ссылки внизу

Таблица 1 — Основные HTML-теги проекта

1.3. CSS — оформление и внешний вид

CSS (Cascading Style Sheets) отвечает за то, как страница выглядит визуально. Без CSS даже хорошо структурированная HTML-страница представляла бы собой просто чёрный текст на белом фоне. CSS задаёт цвета, шрифты, отступы, анимации и расположение элементов. Все стили вынесены в один файл `style.css`, который подключается к каждой странице через тег `<link>`, что является стандартным подходом для многостраничных сайтов. Если нужно поменять, например, основной цвет, достаточно сделать это в одном месте, и изменение сразу отразится на всём сайте.

Flexbox и Grid

2 главных инструмента для раскладки в проекте — Flexbox и CSS Grid. Flexbox хорошо подходит для однострочных раскладок, таких как шапка сайта, навигация, строка с иконкой и текстом, а Grid удобнее там, где нужна сетка из нескольких строк и столбцов: карточки отелей, блок преимуществ, секция с направлениями. Оба инструмента избавляют от необходимости вручную высчитывать отступы и позиционировать каждый элемент.

Цвета и визуальный стиль

Цветовая схема выбрана нейтральная: тёмно-синий (#1e2d3d) и стальной голубой (#4a7fa5) используются как основные цвета, а светло-серый (#f7f8fa) служит фоном страниц. Такая палитра выглядит аккуратно и профессионально, не отвлекая пользователя от содержимого.

Анимации при наведении

Карточки отелей и направлений при наведении курсора плавно поднимаются и получают более глубокую тень. Реализуется это через transition и transform: translateY(-4px). Небольшой эффект, который делает интерфейс более живым и отзывчивым.

Инструмент	Для чего	Где применяется
Flexbox	Выравнивание в строку	Шапка, навигация, кнопка
CSS Grid	Сетка элементов	Карточки отелей и преимуществ
transition	Плавные анимации	Кнопки, карточки, ссылки
linear-gradient	Градиентный фон	Ного-баннер, шапки страниц
@media	Адаптивность	Перестройка под мобильные

Таблица 2 — CSS-инструменты проекта

1.4. JavaScript — логика и интерактивность

JavaScript — это язык программирования, который работает непосредственно в браузере. С его помощью страница перестаёт быть статичной: появляется возможность реагировать на действия пользователя, менять содержимое без перезагрузки, проверять введённые данные и выполнять другие задачи. В данной работе весь JavaScript собран в один файл `main.js`, который подключается в конце каждой страницы. Такой порядок подключения нужен для того, чтобы к моменту выполнения скриптов вся HTML-разметка уже была загружена и доступна.

Работа с DOM

DOM (Document Object Model) — это представление страницы в виде дерева элементов, к которым можно обращаться из JavaScript. Метод `document.getElementById()` позволяет получить нужный элемент по его идентификатору. Свойство `innerHTML` даёт возможность изменить содержимое элемента, а `addEventListener()` позволяет подписаться на события, например клик или изменение поля.

Валидация форм

Перед показом подтверждения бронирования выполняется проверка всех полей формы. Каждое поле проверяется отдельно, и при обнаружении ошибки под соответствующим полем появляется сообщение с её описанием. Страница при этом не перезагружается. Для адреса электронной почты дополнительно проверяется формат с помощью регулярного выражения, так как браузерная проверка не всегда достаточна.

1.5. Как всё это устроено вместе

Сайт состоит из шести файлов: `index.html`, `hotels.html`, `booking.html`, `contacts.html` — это страницы, `style.css` — стили, `main.js` — скрипт. Все страницы подключают один и тот же CSS-файл и один и тот же JS-файл. Благодаря этому не нужно повторять код, ведь, опять же, изменения в стилях или логике сразу отражаются на всём сайте целиком. Структура каждой страницы одинакова: шапка с навигацией, основной контент, подвал. Это создаёт единый вид сайта визуально и помогает пользователю сориентироваться на любой из страниц сайта.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. HTML-разметка страниц

В данном разделе рассмотрены основные HTML-фрагменты проекта с кратким пояснением принятых решений.

Общая шапка и навигация

Шапка одинакова на всех 4 страницах и содержит логотип и навигационное меню. На активной странице ссылка получает класс `active`, чтобы визуально выделить текущий раздел. Используется семантический тег `<header>`, внутри которого расположен `<nav>` с набором ссылок `<a>`.

```
<header class="header">
  <div class="container header__inner">
    <div class="logo">HotelBook</div>
    <nav class="nav">
      <a href="index.html" class="nav__link active">Главная</a>
      <a href="hotels.html" class="nav__link">Отели</a>
      <a href="booking.html" class="nav__link">Бронирование</a>
      <a href="contacts.html" class="nav__link">Контакты</a>
    </nav>
  </div>
</header>
```

Hero-секция с кнопкой перехода в каталог

Главный баннер занимает верхнюю часть главной страницы. Он содержит заголовок, подзаголовок и крупную кнопку «Найти отель», которая переводит пользователя прямо в каталог. Поисковая форма с вводом города и дат была намеренно убрана: без серверной части она не могла полноценно фильтровать результаты. Кнопка реализована тегом `<a>` с атрибутом `href`, указывающим на `hotels.html`, и стилизована через класс `btn--hero`.

```

<section class="hero">
  <div class="container hero__inner">
    <h1 class="hero__title">Найдите идеальный отель</h1>
    <p class="hero__subtitle">Удобное бронирование онлайн</p>
    <a href="hotels.html" class="btn btn--hero">
      Найти отель
    </a>
  </div>
</section>

```

Каталог отелей с фильтрами

Страница каталога состоит из двух колонок: панель фильтров слева и список карточек справа. Фильтры реализованы через стандартные HTML-элементы — 2 числовых поля для диапазона цен, чекбоксы для выбора звёздности и выпадающий список для города. Контейнер карточек `<main id="hotelsList">` изначально пуст и заполняется JavaScript при загрузке страницы.

```

<aside class="filters">
  <h3>Фильтры</h3>

  <div class="filter-group">
    <label>Цена за ночь (₽)</label>
    <div class="price-range">
      <input type="number" id="priceMin" placeholder="От"
value="0" min="0"/>
      <span>—</span>
      <input type="number" id="priceMax" placeholder="До"
value="250" min="0"/>
    </div>
  </div>

  <div class="filter-group">
    <label>Звёздность</label>
    <div class="stars-filter">
      <label><input type="checkbox" value="5" class="star-cb"/>
★★★★★</label>
      <label><input type="checkbox" value="4" class="star-cb"/>
★★★★</label>

```

```

        <label><input type="checkbox" value="3" class="star-cb"/>
        ★★★</label>
        <label><input type="checkbox" value="2" class="star-cb"/>
        ★★</label>
    </div>
</div>

<div class="filter-group">
    <label>Город</label>
    <select id="cityFilter">
        <option value="">Все города</option>
        <option>Баку</option>
        <option>Стамбул</option>
        <option>Дубай</option>
        <option>Москва</option>
    </select>
</div>

    <button class="btn btn--primary" style="width:100%"
onclick="applyFilters()">Применить</button>
    <button class="btn btn--outline" style="width:100%;margin-
top:8px" onclick="resetFilters()">Сбросить</button>
</aside>
<main class="hotels-list" id="hotelsList">

```

Форма бронирования и блок итогов

На странице бронирования расположены 2 части: большая форма слева и компактный блок с итогами справа. Блок итогов обновляется в реальном времени по мере заполнения формы. Под каждым полем находится пустой тег `<span class="error">`, в который JavaScript записывает текст ошибки при неправильном вводе данных.

```

<div class="form-group">
    <label>Email</label>
    <input type="email" id="email"
        placeholder="mail@example.com" />
    <span class="error" id="emailErr"></span>
</div>

<aside class="booking-summary">

```

```

<h3>Итог бронирования</h3>
<div class="summary-row">
  <span>Ночей:</span>
  <span id="sumNights">-</span>
</div>
<div class="summary-row summary-row--total">
  <span>Итого:</span>
  <span id="sumTotal">-</span>
</div>
</aside>

<div class="modal-overlay" id="confirmModal">
  <div class="modal">
    <div class="modal__icon">OK</div>
    <h2>Бронирование подтверждено!</h2>
    <p id="modalText"></p>
    <button onclick="closeModal()">На главную</button>
  </div>
</div>

```

2.2. CSS-стили

Рассмотрим наиболее характерные фрагменты CSS из файла style.css:

Сброс стилей и базовые настройки

В начале файла выполняется универсальный сброс отступов для всех элементов, так как у разных браузеров разные стандартные отступы.

```

*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  color: #2c2c2c;
  background: #f7f8fa;
  line-height: 1.6;
}

```

Шапка с Flexbox

Шапка прилипает к верху экрана при прокрутке (`position: sticky`). Внутри применяется Flexbox с `justify-content: space-between`, чтобы логотип находился слева, а навигация — справа. Активная ссылка визуально выделяется изменением фона и цвета текста.

```
.header {
  background: #fff;
  box-shadow: 0 2px 8px rgba(0,0,0,.07);
  position: sticky;
  top: 0;
  z-index: 100;
}

.header__inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 64px;
}

.nav__link.active {
  background: #eaf2f8;
  color: #4a7fa5;
}
```

Hero-баннер с градиентом

Главный баннер получает тёмный фон через `linear-gradient`. Два цвета плавно переходят друг в друга по диагонали (135 градусов). Белый текст поверх тёмного фона обеспечивает хороший контраст и читаемость.

```
.hero {
  background: linear-gradient(
    135deg,
    #1e2d3d 0%,
    #4a7fa5 100%
  );
}
```

```
padding: 72px 0 56px;
color: #fff;
}
```

Карточки отелей с анимацией

Карточки отелей используют Flexbox: изображение слева, текст справа. При наведении карточка поднимается на 4 пикселя вверх, а тень становится глубже. Это реализуется через transition и псевдокласс :hover. Такое поведение визуально подсказывает пользователю, что карточка кликабельна.

```
.hotel-card {
  background: #fff;
  border-radius: 10px;
  box-shadow: 0 2px 12px rgba(0,0,0,.06);
  display: flex;
  overflow: hidden;
  transition: box-shadow .2s;
}

.hotel-card:hover {
  box-shadow: 0 6px 20px rgba(0,0,0,.11);
}

.hotel-card__img {
  width: 220px;
  min-height: 160px;
  object-fit: cover;
}
```

Адаптивность через @media

На мобильных устройствах двухколоночные макеты перестраиваются в одноколоночные. Медиа-запрос срабатывает при ширине экрана менее 700 пикселей, что соответствует большинству современных смартфонов.

```
@media (max-width: 700px) {
  .hotels-page {
    grid-template-columns: 1fr;
  }
}
```

```
}  
.booking-page {  
  grid-template-columns: 1fr;  
}  
}
```

2.3. JavaScript-логика

Весь JavaScript находится в файле `main.js`. Рассмотрим его ключевые функции:

Данные об отелях

Все отели хранятся в массиве объектов в начале файла. Каждый объект содержит название, количество звёзд, город, цену, ссылку на фотографию и описание. Такой подход удобен: чтобы добавить новый отель, достаточно добавить один объект в массив, не затрагивая HTML-разметку.

```
const hotels = [  
  {  
    id: 1,  
    name: 'Grand Baku Hotel',  
    stars: 5,  
    city: 'Баку',  
    price: 205,  
    img: 'https://images.unsplash.com/...',  
    desc: 'Роскошный отель в центре Баку.'  
  },  
  {  
    id: 8,  
    name: 'Budget Express Moscow',  
    stars: 2,  
    city: 'Москва',  
    price: 51,  
    img: 'https://images.unsplash.com/...',  
    desc: 'Небольшой отель рядом с метро.'  
  },  
];
```


Генерация карточек через renderHotels()

Функция `renderHotels()` принимает массив отелей и генерирует из него HTML-карточки, вставляя их в контейнер на странице. Применяется метод `map()`, который обходит каждый элемент массива и возвращает строку с HTML. Результат соединяется через `join("")` и записывается в `innerHTML` контейнера. Если массив пуст, то выводится сообщение о том, что ничего не найдено.

```
function renderHotels(list) {
  const container = document.getElementById('hotelsList');
  if (!container) return;

  if (list.length === 0) {
    container.innerHTML =
      '<p>По вашему запросу отели не найдены.</p>';
    return;
  }

  container.innerHTML = list.map(h => `
    <div class='hotel-card'>
      <img class='hotel-card__img'
        src='${h.img}' alt='${h.name}' />
      <div class='hotel-card__body'>
        <span class='hotel-card__name'>${h.name}</span>
        <div class='hotel-card__price'>
          ${h.price}m <span>/ ночь</span>
        </div>
        <a href='booking.html' class='btn btn--primary'>
          Забронировать
        </a>
      </div>
    </div>
  `).join('');
}
```

Фильтрация отелей через applyFilters()

Фильтрация работает следующим образом: считываются текущие значения всех фильтров, после чего применяется метод `.filter()` к исходному

массиву `hotels`. Этот метод возвращает новый массив, содержащий только те элементы, для которых условие вернуло `true`. Результат передаётся в `renderHotels()`, и карточки обновляются. Очень важно отметить, что фильтрация всегда применяется к исходному массиву, а не к уже отфильтрованному результату.

```
function applyFilters() {
  const minPrice = parseInt(
    document.getElementById('priceMin').value) || 0;
  const maxPrice = parseInt(
    document.getElementById('priceMax').value) || 9999;
  const city = document.getElementById('cityFilter').value;

  const checkedStars = [
    ...document.querySelectorAll('.star-cb:checked')
  ].map(cb => parseInt(cb.value));

  const filtered = hotels.filter(h => {
    const priceOk = h.price >= minPrice &&
      h.price <= maxPrice;
    const cityOk = !city || h.city === city;
    const starsOk = checkedStars.length === 0 ||
      checkedStars.includes(h.stars);
    return priceOk && cityOk && starsOk;
  });

  renderHotels(filtered);
}
```

Валидация формы бронирования

Перед показом модального окна подтверждения функция `submitBooking()` проверяет каждое поле. Переменная `valid` изначально равна `true`, и если хотя бы одна проверка не прошла, она принимает значение `false` — в этом случае функция завершается без показа подтверждения. Email проверяется отдельно через регулярное выражение. Дополнительно

проверяется логика дат: дата выезда не должна совпадать с датой заезда или предшествовать ей.

```
function submitBooking() {
  let valid = true;

  const firstName = document.getElementById('firstName');
  const errFirst = document.getElementById('firstNameErr');
  errFirst.textContent = '';
  if (!firstName.value.trim()) {
    errFirst.textContent = 'Поле «Имя» обязательно';
    valid = false;
  }

  const email = document.getElementById('email');
  const emailRe = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRe.test(email.value)) {
    document.getElementById('emailErr').textContent =
      'Введите корректный email';
    valid = false;
  }

  const ci = document.getElementById('bookCheckIn').value;
  const co = document.getElementById('bookCheckOut').value;
  if (ci && co && ci >= co) {
    document.getElementById('checkOutErr').textContent =
      'Дата выезда должна быть позже заезда';
    valid = false;
  }

  if (!valid) return;

  document.getElementById('confirmModal')
    .classList.add('active');
}
```

Автоматический пересчёт итоговой суммы

Функция `updateSummary()` вызывается при каждом изменении значений в полях бронирования. Она рассчитывает количество ночей как разность дат в миллисекундах, переведённую в дни, умножает на цену выбранного отеля и обновляет итоговый блок справа от формы.

```
function updateSummary() {
  const hotelKey = document.getElementById('hotelSelect').value;
  const checkIn  = document.getElementById('bookCheckIn').value;
  const checkOut = document.getElementById('bookCheckOut').value;

  let nights = 0;
  if (checkIn && checkOut && checkOut > checkIn) {
    const ms = 1000 * 60 * 60 * 24;
    nights = Math.round(
      (new Date(checkOut) - new Date(checkIn)) / ms
    );
  }

  document.getElementById('sumNights').textContent =
    nights > 0 ? nights : '-';

  if (hotelKey && nights > 0) {
    const price = hotelPrices[hotelKey];
    document.getElementById('sumTotal').textContent =
      (price * nights) + 'm';
  }
}
```

ГЛАВА 3. РЕЗУЛЬТАТЫ РАБОТЫ

По результатам разработки получился рабочий четырёхстраничный сайт с единым дизайном и навигацией. Рассмотрим наглядно, что реализовано на каждой из страниц.

3.1. Главная страница

Главная страница открывается крупным Hero-баннером с градиентным тёмно-синим фоном. По центру расположена большая кнопка «Найти отель», при нажатии на которую пользователь переходит прямо в каталог. Поисковая форма с вводом города и дат была убрана по простой причине: без серверной части она не могла фильтровать результаты, поэтому вместо неё оставлена простая и понятная точка входа в каталог.

Ниже расположены два блока: «Почему выбирают нас» (4 карточки преимуществ) и «Популярные направления» (4 карточки городов с реальными фотографиями). Количество отелей в карточках направлений соответствует реальному каталогу: Баку — 3, Стамбул — 2, Дубай — 1, Москва — 2.

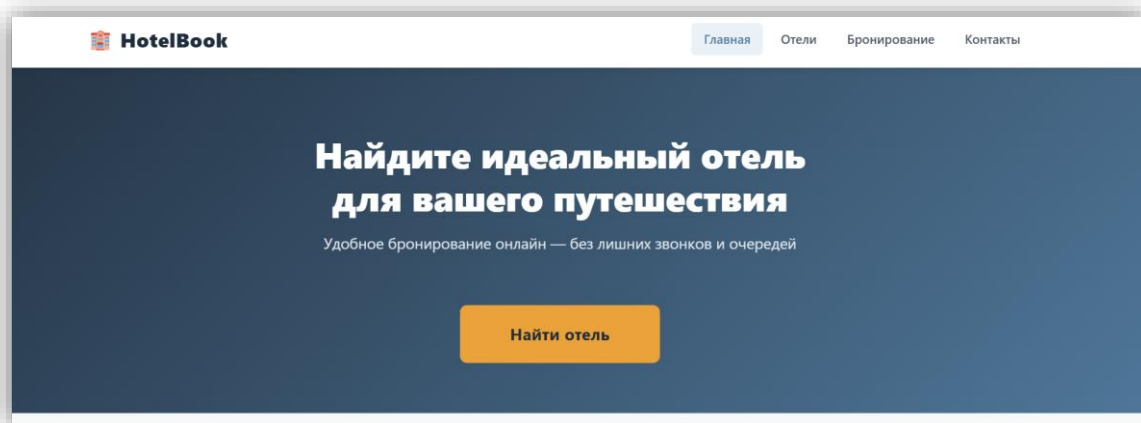


Рисунок 1 — Главная страница сайта HotelBook

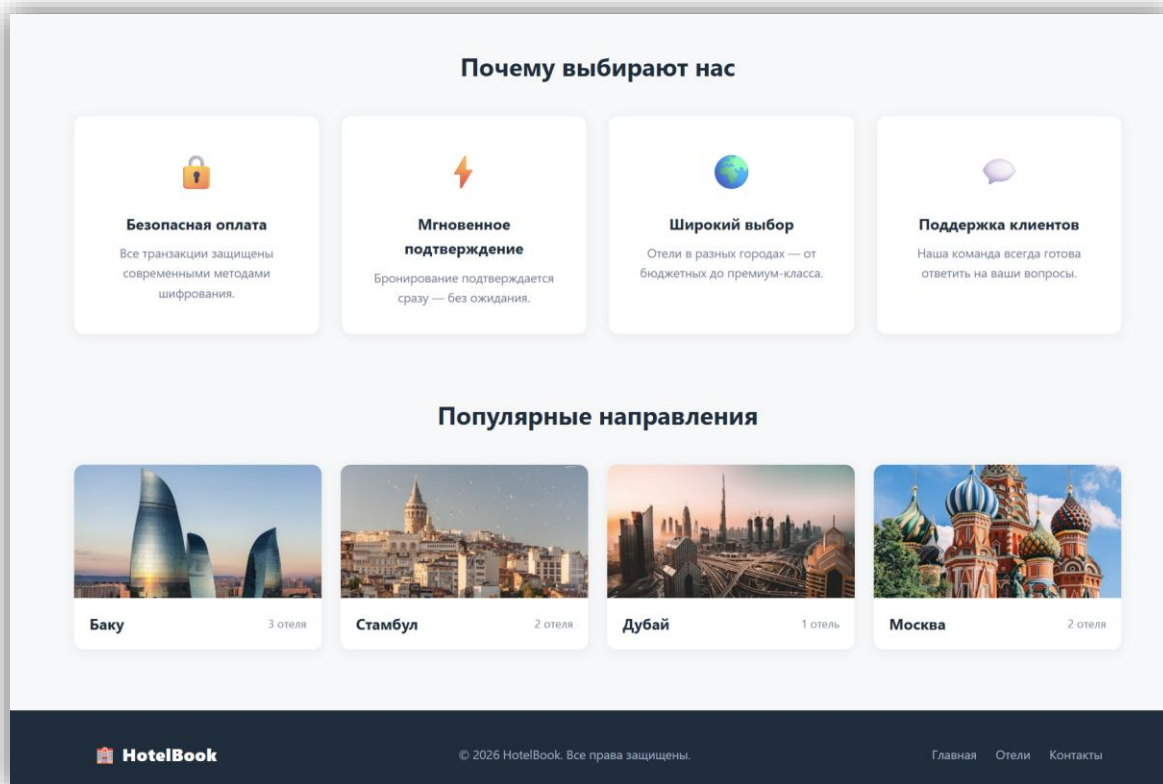


Рисунок 2 — Блоки преимуществ и популярных направлений

3.2. Каталог отелей

Каталог построен по двухколоночному макету: слева панель фильтров, справа список карточек. Восемь отелей разбиты по четырём городам и охватывают диапазон от двух до пяти звёзд. Самый доступный вариант — двухзвёздочный Budget Express Moscow за 51₽/ночь. Наиболее дорогой — пятизвёздочный Istanbul Plaza за 255₽/ночь.

Каждая карточка содержит фотографию отеля, название, звёздность, город, краткое описание, цену и кнопку перехода к бронированию. При применении фильтров список обновляется мгновенно без перезагрузки страницы.

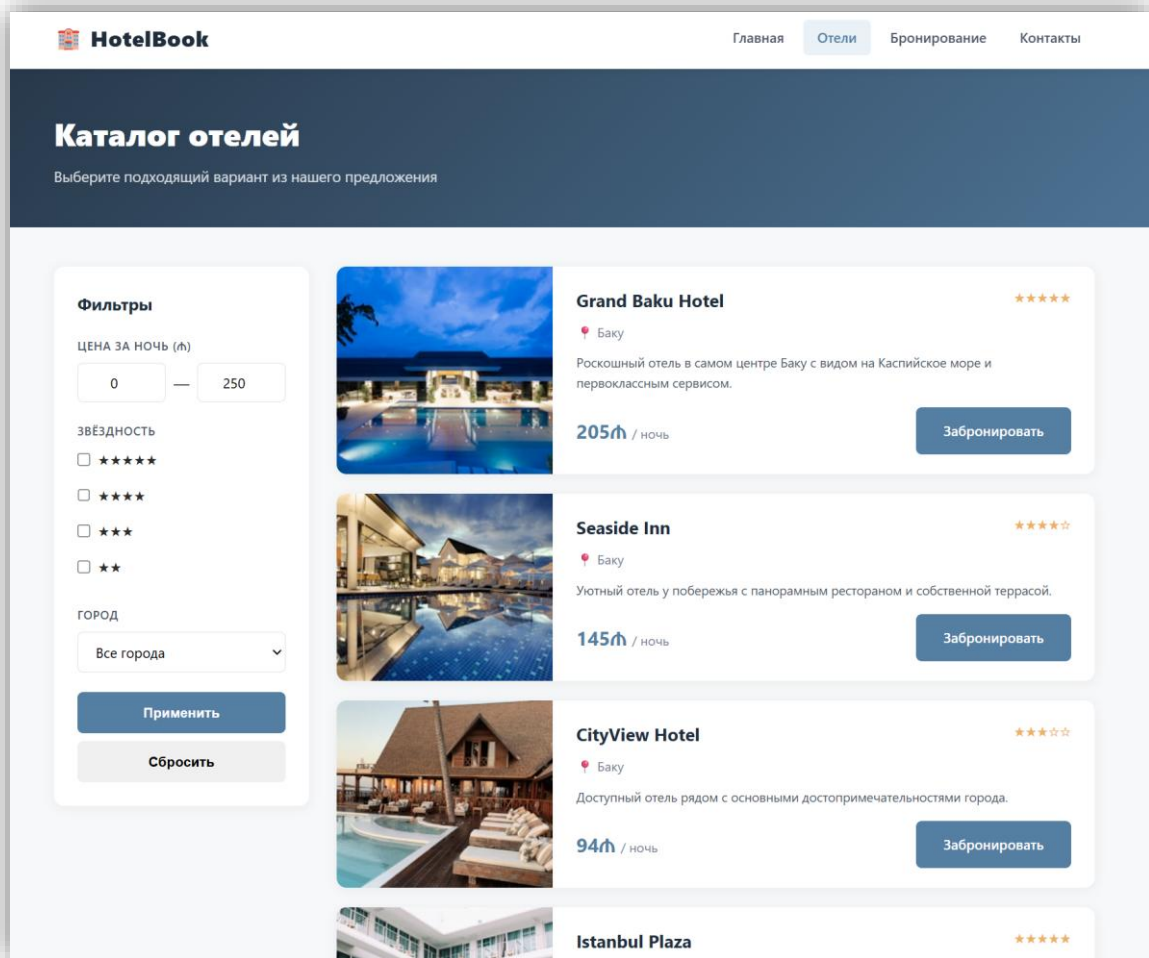


Рисунок 3 — Каталог отелей с карточками и панелью фильтров

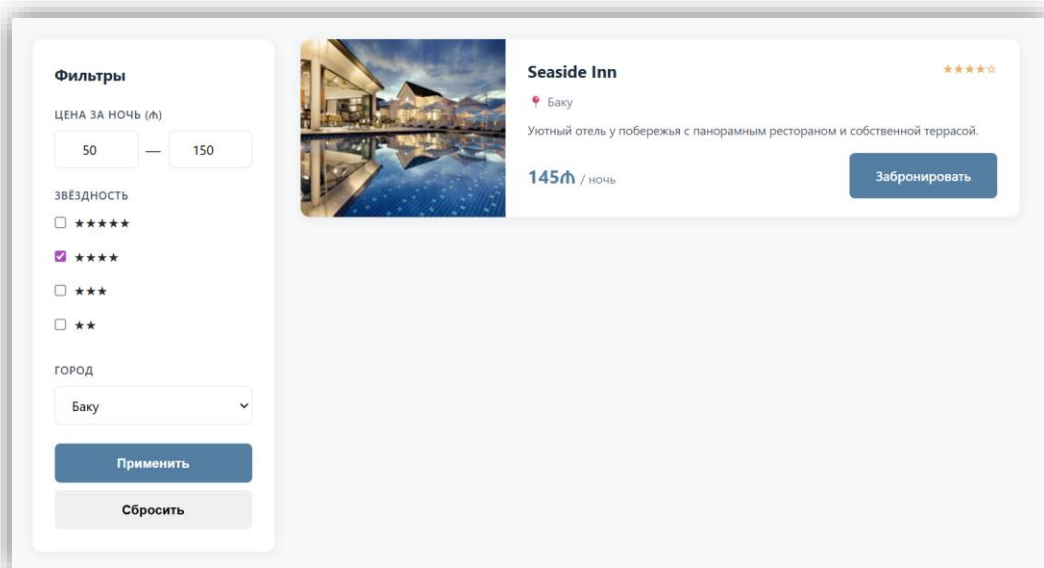


Рисунок 4 — Результат работы фильтрации

3.3. Страница бронирования

Страница бронирования содержит форму с 7 обязательными полями и блок итогов, который обновляется в реальном времени. Как только выбран отель и введены даты, система рассчитывает количество ночей и отображает итоговую стоимость в манатах. При попытке отправить форму с ошибками под каждым проблемным полем появляется поясняющее сообщение о том, в чём конкретно ошибка. После успешного заполнения открывается модальное окно с персонализированным подтверждением, в котором указаны имя пользователя и название выбранного отеля.

The screenshot displays a booking interface with two main sections: 'Ваши данные' (Your data) and 'Итог бронирования' (Booking summary).

Ваши данные (Your data):

- ИМЯ (Name):** Input field with placeholder 'Имя'.
- ФАМИЛИЯ (Surname):** Input field with placeholder 'Фамилия'.
- EMAIL:** Input field with placeholder 'mail@example.com'.
- ТЕЛЕФОН (Phone):** Input field with placeholder '+994 55 319 67 93'.

Детали бронирования (Booking details):

- ВЫБЕРИТЕ ОТЕЛЬ (Select hotel):** Dropdown menu showing 'Grand Baku Hotel ★★★★★ — 205лв/ночь'.
- ДАТА ЗАЕЗДА (Check-in date):** Input field showing '12.05.2026' with a calendar icon.
- ДАТА ВЫЕЗДА (Check-out date):** Input field showing '14.05.2026' with a calendar icon.
- КОЛИЧЕСТВО ГОСТЕЙ (Number of guests):** Dropdown menu showing '1'.
- ОСОБЫЕ ПОЖЕЛАНИЯ (НЕОБЯЗАТЕЛЬНО) (Special requests (optional)):** Text area containing 'Провести уборку номера до заезда в отель.' (Clean the room before check-in).
- Подтвердить бронирование (Confirm booking):** Blue button at the bottom.

Итог бронирования (Booking summary):

Отель:	Grand Baku Hotel ★★★★★
Заезд:	12 мая 2026 г.
Выезд:	14 мая 2026 г.
Ночей:	2
Гостей:	1
Итого:	410лв

Рисунок 5 — Страница бронирования с расчётом стоимости

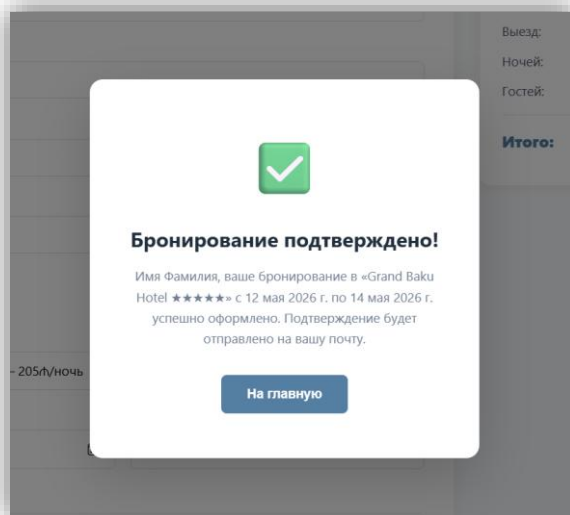


Рисунок 6 — Модальное окно подтверждения

Ваши данные

ИМЯ

Поле «Имя» обязательно для заполнения

ФАМИЛИЯ

Поле «Фамилия» обязательно для заполнения

EMAIL

Введите корректный email

ТЕЛЕФОН

Поле «Телефон» обязательно для заполнения

Детали бронирования

ВЫБЕРИТЕ ОТЕЛЬ

— Выберите отель —

Поле «ОТЕЛЬ» обязательно для заполнения

ДАТА ЗАЕЗДА

Поле «Заезд» обязательно для заполнения

ДАТА ВЫЕЗДА

Поле «Выезд» обязательно для заполнения

КОЛИЧЕСТВО ГОСТЕЙ

1

ОСОБЫЕ ПОЖЕЛАНИЯ (НЕОБЯЗАТЕЛЬНО)

Подтвердить бронирование

Итог бронирования

ОТЕЛЬ: —

Заезд: —

Выезд: —

Ночей: —

Гостей: —

Итого: —

Рисунок 7 — Пример работы валидации формы

3.4. Страница контактов

На странице контактов 2 блока расположены в 2 колонках. В левом блоке размещена контактная информация: адрес, телефон, email и режим работы (понедельник — пятница, 9:00 — 18:00). Правый блок содержит форму обратной связи с полями для имени, email, темы обращения и текста сообщения. При успешной отправке форма очищается и появляется зелёное уведомление.

The screenshot displays a web interface for 'Contacts'. The header is dark blue with the title 'Контакты' and the tagline 'Мы всегда рады помочь вам'. The main content is divided into two columns. The left column, titled 'Свяжитесь с нами', lists contact details: 'Адрес' (г. Баку, ул. Низами, 42, офис 8), 'Телефон' (+994 12 555 00 00), 'Email' (support@hotelbook.az), and 'Режим работы' (Пн–Пт: 9:00–18:00). The right column, titled 'Написать нам', contains a form with fields for 'ВАШЕ ИМЯ', 'EMAIL', and 'ТЕМА' (with a dropdown menu showing 'Другое'). Below these is a large text area for 'СООБЩЕНИЕ'. A blue button 'Отправить сообщение' is at the bottom of the form. A green notification box at the bottom right states: '✓ Ваше сообщение отправлено! Мы ответим в течение рабочего дня.'

Рисунок 8 — Страница контактов

3.5. Итоговая оценка результатов

По идее, все задачи по созданию сайта успешно выполнены: все 4 страницы связаны единой навигацией и воспринимаются как цельный сайт за счёт общего файла стилей. Карточки отелей отображаются из массива данных, и для добавления нового отеля достаточно дополнить массив одним объектом, не затрагивая разметку. Фильтрация работает в реальном времени, форма бронирования валидируется с понятными сообщениями об ошибках, а итоговая стоимость пересчитывается автоматически при выборе отеля и дат.

Отдельно стоит отметить персонализированное модальное окно подтверждения. После успешной отправки формы пользователь видит своё имя и название выбранного отеля в тексте подтверждения, хоть это и мелочь, но зато как заметно улучшает общее восприятие интерфейса! Сам сайт визуально был выполнен в сдержанной цветовой схеме без всяких ярких элементов.

Кроме того, сам сайт можно посмотреть на своём же устройстве, перейдя по ссылке — <https://saidu-u.github.io/Development-of-an-online-hotel-booking-system/>.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы мы смогли создать многостраничный сайт системы онлайн-бронирования отелей с использованием HTML, CSS и JavaScript. Эта работа позволила лучше разобраться во взаимодействии 3 клиентских технологий, отсюда мы наглядно поняли, что HTML задаёт структуру, CSS отвечает за внешний вид, а JavaScript добавляет интерактивность и поведение.

В процессе разработки пришлось решить парочку непростых задачек: это организовать данные об отелях в удобную структуру, написать логику фильтрации, продумать валидацию форм и ещё реализовать динамический пересчёт стоимости бронирования. Всё это показало, что клиентская разработка даже без серверной части уже требует хорошего подхода к написанию кода. Сайт в итоге функционирует как надо: пользователь может найти отель, отфильтровать список по нужным параметрам и оформить бронирование, получив подтверждение.

В будущем можно будет рассмотреть добавление серверной части с реальной базой данных, авторизацию пользователей, интеграцию с картами и даже полноценную систему оплаты. Но это уже уровень явно выше, который выходит за рамки данной работы.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. MDN Web Docs. Веб-технологии для разработчиков (2025). Mozilla Foundation. — <https://developer.mozilla.org/ru/docs/Web>
2. W3Schools. HTML, CSS, JavaScript Tutorials (2025). Refsnes Data. — <https://www.w3schools.com>
3. CSS-Tricks. A Complete Guide to Flexbox (2025). CSS-Tricks. — <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
4. CSS-Tricks. A Complete Guide to CSS Grid (2025). CSS-Tricks. — <https://css-tricks.com/snippets/css/complete-guide-grid/>
5. WHATWG. HTML Living Standard (2025). Web Hypertext Application Technology Working Group. — <https://html.spec.whatwg.org/>
6. Bnovo. Система онлайн-бронирования отелей (2024). — <https://bnovo.ru/blog/hotel-online-booking-system/>
7. SCAND. Как разработать систему онлайн-бронирования для бизнеса (2023). — <https://scand.com/ru/company/blog/how-to-develop-your-own-web-booking-system/>
8. Спасов Д.Ю. Веб-приложение система онлайн-бронирования для гостиничного бизнеса (2024). Калужский государственный университет им. К.Э. Циолковского. — <https://cyberleninka.ru/article/n/veb-prilozhenie-sistema-onlayn-bronirovaniya-dlya-gostinichnogo-biznesa>